

Agente IA para *matching* paciente–ensayo clínico

Construid un agente que, dado el perfil de un paciente, identifique, evalúe y ordene (*ranking*) ensayos clínicos relevantes de forma razonada y automatizada.

Contexto

El reclutamiento ineficiente es una de las principales causas de fracaso de los ensayos clínicos. Miles de pacientes elegibles nunca llegan a conocer los estudios disponibles. Este reto os pide construir un sistema agéntico que razone sobre criterios médicos complejos, consulte bases de datos reales y genere recomendaciones justificadas. No simplemente tecnología RAG (*Generación Aumentada por Recuperación*): el agente debe planificar, filtrar, razonar y sintetizar usando múltiples herramientas.

Tareas del agente

<u>1. Retrieval estratégico</u> Búsqueda en ClinicalTrials.gov con <i>queries</i> encadenadas y refinadas, sin saturar las etapas siguientes.	<u>2. Verificación de elegibilidad</u> Evaluar criterio a criterio: <i>met / not met / not enough info</i> . Negaciones, umbrales, lógica temporal.	<u>3. Ranking razonado</u> Diseñar y aplicar una función de <i>scoring</i> que priorice ensayos según elegibilidad, fase y estado de reclutamiento.
<u>4. Información faltante</u> Formular la pregunta clínica exacta que resuelve cada criterio indeterminado, no simplemente marcarlo.	<u>5. Dossier de preselección</u> Generar un documento estructurado por ensayo (resumen, tabla de criterios, flags de atención) listo para revisión médica.	<u>6. Monitorización continua*</u> Detectar cambios en ensayos (estado, criterios) e identificar qué pacientes de una cohorte se ven afectados. *(<u>opcional</u>)

Datos y acceso

- **ClinicalTrials.gov ; fuente de ensayos**

API REST con +500.000 estudios registrados. Sin autenticación. Soporta filtrado por condición, fase, estado de reclutamiento y localización geográfica. Usad *endpoints* clave como GET /studies.

API explorer: <https://clinicaltrials.gov/data-api/api>

Documentación: <https://clinicaltrials.gov/data-api/about-api>

- **TREC Clinical Trials 2021 & 2022; benchmark principal**

El benchmark de referencia para este problema con dataset que contiene perfiles sintéticos de pacientes con juicios de relevancia graduada sobre ensayos reales de ClinicalTrials.gov. Métrica oficial: NDCG@10. Descarga libre, sin registro.

Portal TREC CT: <https://trec.nist.gov/data/trials.html>

- **MeSH ; normalización clínica**

Para resolver sinónimos médicos y mapear términos del perfil del paciente a conceptos estándar. Se pueden usar otras ontologías médicas también.

Frameworks agénticos sugeridos

LangGraph: Control explícito del flujo del agente mediante grafos de estados. Ideal para modelar el ciclo retrieval → verificación → ranking con ramas condicionales y memoria entre pasos.

LlamaIndex: Especializado en pipelines de indexación y recuperación sobre documentos. Útil para construir el módulo de búsqueda semántica sobre criterios de elegibilidad y guías clínicas.

n8n: Orquestación visual de flujos con nodos de IA, HTTP y transformación de datos. Buena opción para prototipar el pipeline completo o para equipos que quieran iterar rápido sobre la arquitectura.

Pydantic AI: Agentes con salidas estructuradas y validación estricta de tipos. Muy útil para garantizar que las evaluaciones de criterios (met/not met/NEI) y el ranking sean parseables y reproducibles.

No hay restricción de *framework*. Podéis combinar varios o construir sin *framework* si lo justificáis. Lo que se evalúa es el comportamiento del agente, no la tecnología subyacente.

Para los modelos de lenguaje (**LLMs**) subyacentes, podéis consultar en la universidad qué prestaciones tenéis con cuentas de estudiante o usar *free tier* de proveedores. Podéis usar también modelos en local (ejemplo: Ollama). En resumen, sin restricción de modelo. Es obligatorio documentar qué modelo, versión y configuración de inferencia habéis empleado.

Estructura de evaluación

Autoevaluación sobre TREC: usad los datos públicos de TREC para medir vuestro agente durante el desarrollo. Las métricas obtenidas deben reportarse en la memoria para cada tarea implementada. Son la evidencia principal de que el sistema funciona y generaliza.

$$\text{Score} = 0.20 \times \text{Recall@20 (T1)} + 0.30 \times \text{Micro-F1 (T2)} + 0.25 \times \text{NDCG@10 (T3)} + 0.15 \times \text{calidad preguntas (T4)} + 0.10 \times \text{completitud dossier (T5)}$$

Recall@20: cobertura de ensayos relevantes; Micro-F1: exactitud por criterio met/not met/NEI; NDCG@10: calidad del ranking

Parte del reto es entender qué mide cada métrica, por qué es la adecuada para cada tarea y qué decisiones de diseño del agente las afectan. La nota final no depende únicamente de los números: el equipo docente evaluará también la solidez de la arquitectura y el análisis de errores a partir de los entregables. También dispondrá de un nuevo data set de evaluación para capturar la capacidad de generalización.

Entregables

- **Repositorio Git** con el código del agente y un README que permita reproducir los resultados desde cero: dependencias, configuración y comandos de ejecución.
- **Memoria técnica** (máx. 5 páginas): arquitectura del agente, herramientas implementadas, métricas obtenidas en TREC por tarea, análisis de errores y comparación con al menos un baseline de los publicados en TREC.
- **Fichero de predicciones** sobre el conjunto de evaluación en el formato especificado (JSON). El agente debe poder ejecutarse de forma no interactiva sobre una lista de perfiles de paciente.